Reinforcement Learning for Strategy Enhancement in Real-Time Strategy Games

Exam Roll: 202192 & 202220
Computer Science and Engineering
Jahangirnagar University

Background

The RTS Challenge:

- **Dynamic Environments:** Constant change and unpredictable opponent actions.
- **Vast Action Spaces:** Numerous simultaneous decisions (e.g., unit control, resource management, construction).
- **Real-time Demands:** Rapid decision-making within strict time constraints.

Reinforcement Learning: A Powerful Solution

- **Adaptability:** Learns optimal strategies through continuous game interactions.
- **Robustness:** Handles uncertainty and adapts to unforeseen events.
- **Continuous Improvement:** Refines strategies based on game outcomes and feedback.



Visual Representation of a Complex RTS Game

Motivation

RTS Games: A Crucible for AI Research

Advance AI: Develop strategic and adaptive AI agents capable of handling complex scenarios and making real-time decisions.

Bridge the Gap: Close the performance gap between AI and human players by creating AI that can compete with or surpass human strategies.

Real-world Applications:

- **Military Simulations:** Enhance strategic planning and battlefield simulations.
- **Resource Management:** Improve allocation and utilization of resources in various industries.
- **Robotics:** Enable robots to make autonomous decisions in dynamic environments.

Literature Review

- Mnih et al.[1] **Deep Q-Networks (DQN)** was introduced, combining deep learning with reinforcement learning (RL) to achieve **human-level performance** in Atari games. This laid the foundation for RL applications in complex environments.
- Ontañón et al.[2] conducted a **survey of AI research** in RTS games, highlighting the **limitations of rule-based systems** and the potential of RL to address dynamic and adaptive challenges.
- Silver et al.[3] developed **AlphaZero**, demonstrating the power of self-play and RL in mastering games like chess, shogi, and Go. It showcases how RL can drive **strategic advancements** in gaming.
- Huang et al.[4] proposed **Gym-μRTS**, a lightweight platform for RL research in RTS games, emphasizing **cost-effective training** and **evaluation** for RL agents.

Problem Formulation

Real-time strategy (RTS) games present unique challenges for AI systems, including:

- Managing large state spaces with thousands of possible game scenarios at any given moment.
- Handling high-dimensional action spaces involving simultaneous unit control, resource allocation, and strategic planning.
- Adapting to dynamic, partially observable environments where strategies must evolve in real time.

Reinforcement Learning (RL) addresses these challenges by enabling AI agents to learn optimal behaviors through iterative interactions with the environment. The core objective is to identify the optimal policy π^* that maximizes the agent's expected cumulative reward:

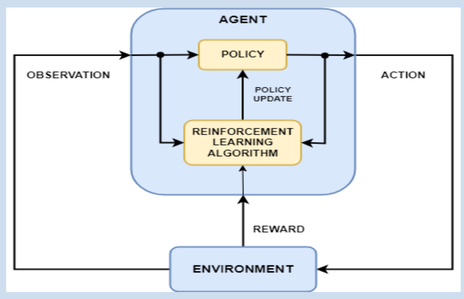
$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r_t}\right]$$

where:

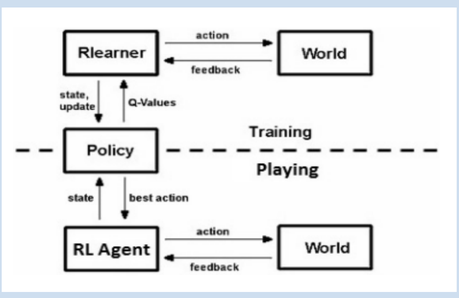
- π: The policy mapping states to actions.
- γ: Discount factor for future rewards ($0 \leq \gamma \leq 1$).
- $\mathbf{r_t}$: Reward received at time step t.

Framework

- **State Encoding:**
  - Game State Representation: Neural networks capture game dynamics (e.g., resource management, unit control).
- **Action Selection:**
  - Discrete/Continuous Actions: Define action sets (e.g., unit movements, resource allocation).
- **Learning Framework:**
  - Policy & Algorithm: Neural network-based policy combined with RL algorithms (DQN, Policy Gradient).
- **Performance Optimization:**
  - Reward Design: Shape rewards for strategic decisions (e.g., combat victories, efficient resource use).
  - Strategic Balancing:
    - Exploration-Exploitation Trade-off: Carefully balance exploration (discovering new strategies) with exploitation (leveraging known effective actions) using techniques like epsilon-greedy to maximize long-term rewards.



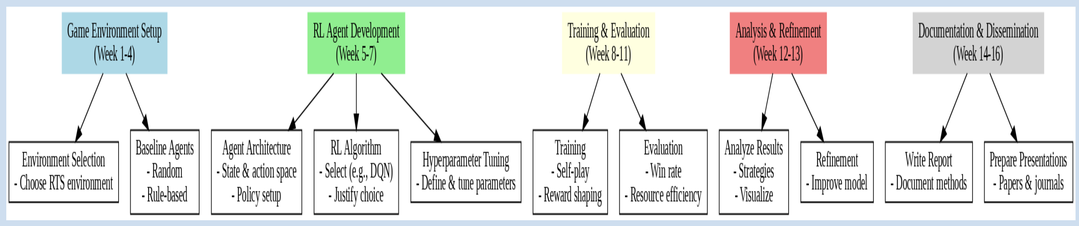
Reinforcement Learning Process



Reinforcement Learning Architecture

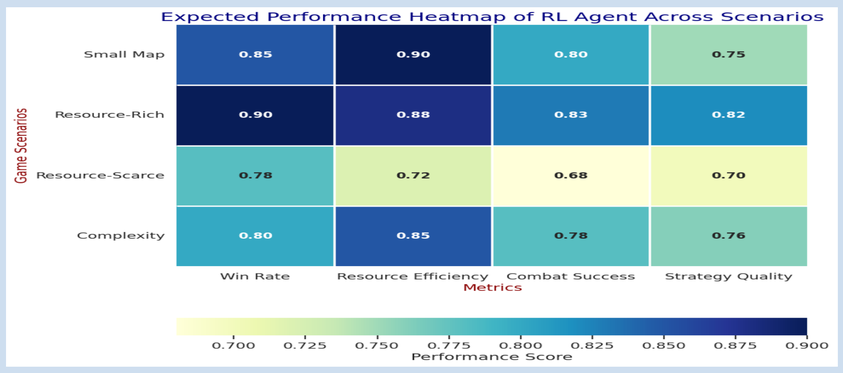
Proposed Methodology

1. **Game Environment :** Develop an RTS environment with core mechanics and baseline agents for comparison.
2. **RL Agent Development :** Design the RL agent and optimize algorithms with hyperparameter tuning.
3. **Training and Evaluation :** Train with reward shaping, apply exploration strategies, and evaluate performance (e.g., **win rate**).
4. **Analysis and Refinement :** Analyze results, **refine architecture**, and retrain for improvement.



Progress Timeline

Expected Results



Expected Performance Heatmap

- **Performance Analysis:** Heatmap visualizes agent performance across diverse scenarios (e.g., small maps, resource-rich environments) and metrics (win rate, resource efficiency, combat success, strategy quality).
- **Actionable Insights:** Enables identification of strengths, weaknesses, and performance trends to guide future research and refinements.

Reference

1. V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, and D. Hassabis. Human-level control through deep reinforcement learning. Nature, 518(7540):529–533, 2015
2. S. Ontañón, G. Synnaeve, A. Uriarte, F. Richoux, D. Churchill, and M. Preuss. A survey of real-time strategy game AI research and competition in StarCraft. IEEE Transactions on Computational Intelligence and AI in Games, 5(4):293–311, 2013.
3. D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362(6419):1140–1144, 2018.
4. S. Huang, S. Ontañón, C. Bamford, and L. Grela. Gym-μRTS: Toward affordable full-game real-time strategy games research with deep reinforcement learning. In 2021 IEEE Conference on Games